

1. Estratégia de Divisão e Conquista (*Divide-and-Conquer*)

A estratégia de **Divisão e Conquista** é uma abordagem recursiva para projeto de algoritmos. Ela resolve um problema dividindo-o em subproblemas menores semelhantes ao problema original, resolvendo esses subproblemas recursivamente e, em seguida, combinando suas soluções para resolver o problema principal.

A abordagem segue **três passos essenciais** em cada nível da recursão:

1. **Dividir (*Divide*)**: Divide o problema original em um ou mais subproblemas menores que são instâncias em menor escala do mesmo problema.
2. **Conquistar (*Conquer*)**: Resolve os subproblemas recursivamente. Se o tamanho do subproblema for pequeno o suficiente (caso base), ele é resolvido de forma direta/trivial.
3. **Combinar (*Combine*)**: Combina as soluções dos subproblemas para construir a solução do problema original.

2. O Algoritmo Merge Sort

O **Merge Sort** é um algoritmo de ordenação clássico fundamentado na técnica de divisão e conquista. Para ordenar um arranjo de n elementos, o algoritmo opera com a abordagem da divisão e conquista.

A Operação Principal (**MERGE**)

O trabalho computacional principal ocorre na etapa de combinação, realizada pelo procedimento auxiliar **MERGE**. Como as duas metades já estão ordenadas, o procedimento as percorre simultaneamente, comparando os elementos das pontas e construindo a sequência final ordenada em tempo linear.

Complexidade e Análise Assintótica

Melhor, pior e médio com mesma complexidade $O(n \log n)$

```
// Função auxiliar para intercalar (merge) duas metades ordenadas
void merge(int arr[], int inicio, int meio, int fim) {
    int n1 = meio - inicio + 1;
    int n2 = fim - meio;

    // Cria vetores temporários para as duas metades
    int* esquerda = new int[n1];
    int* direita = new int[n2];

    // Copia os dados para os vetores temporários
    for (int i = 0; i < n1; i++) {
        esquerda[i] = arr[inicio + i];
    }
    for (int j = 0; j < n2; j++) {
        direita[j] = arr[meio + 1 + j];
    }
}
```

```
// Intercala os vetores temporários de volta em arr[inicio..fim]
int i = 0;      // Índice inicial da primeira metade
int j = 0;      // Índice inicial da segunda metade
int k = inicio; // Índice inicial do vetor combinado

while (i < n1 && j < n2) {
    if (esquerda[i] <= direita[j]) {
        arr[k] = esquerda[i];
        i++;
    } else {
        arr[k] = direita[j];
        j++;
    }
    k++;
}

// Copia os elementos restantes de esquerda[], se houver
while (i < n1) {
    arr[k] = esquerda[i];
    i++;
    k++;
}

// Copia os elementos restantes de direita[], se houver
while (j < n2) {
    arr[k] = direita[j];
    j++;
    k++;
}

// Libera a memória alocada dinamicamente
delete[] esquerda;
delete[] direita;
}

// Função principal do Merge Sort
void mergeSort(int arr[], int inicio, int fim) {
    // Caso base: sub-arranjo de tamanho 0 ou 1 já está ordenado
    if (inicio >= fim) {
        return;
    }

    // Evita overflow de (inicio + fim) / 2
    int meio = inicio + (fim - inicio) / 2;

    // Divide e conquista: ordena as duas metades
    mergeSort(arr, inicio, meio);
    mergeSort(arr, meio + 1, fim);

    // Combina as duas metades ordenadas
    merge(arr, inicio, meio, fim);
}
```